

S1: Tool Survey & Proposal / SLO Report

Companion document to [Report.md](#) (in [01_Report/](#)). Part A consolidates the tool survey; Part B consolidates the SLO results. Both align with Report.md Sections 6, 7.6, 8.1, and 11.

Part A: Tool Survey & Proposal

Covers the S1 requirement: at least 3 candidate tools (1 traditional, 1 AI direction, 1 fallback), scored across 5 criteria.

Candidates considered: k6 (selected as primary), Apache JMeter (selected as secondary), Locust (considered as a fallback, not chosen), and an LLM-assisted (ChatGPT/Claude) scripting workflow (selected as the required AI-assisted direction).

Comparison Matrix (5 Criteria)

Criteria	k6 (primary, chosen)	Apache JMeter (secondary, chosen)	Locust (fallback, not chosen)	AI-assisted scripting (chosen as the AI direction)
Licensing cost	Free, open source (AGPL v3)	Free, open source (Apache License 2.0)	Free, open source (MIT)	Not free by itself, costs whatever the underlying AI subscription or API used to draft scripts costs
Learning curve	Moderate: needs JavaScript, but the syntax is close to plain scripting and the CLI summary is easy to read	Low to moderate: the GUI lets a newcomer build a test plan by dragging elements, but the underlying XML test plan gets harder to read as it grows	Moderate to high: needs Python plus some familiarity with its concurrency model to write a realistic locustfile	Low to get a first draft out, but real testing expertise is still needed to judge whether that draft is correct

Criteria	k6 (primary, chosen)	Apache JMeter (secondary, chosen)	Locust (fallback, not chosen)	AI-assisted scripting (chosen as the AI direction)
Fit with EShop	Strong: JS scripting sits close to EShop's own Node.js/Express stack, and k6's SharedArray parameterization map directly onto EShop's login flow and rate limiter	Strong: the mature HTTP Sampler and JSON Extractor handle EShop's REST API and JWT flow well, and its non-GUI mode is what the team actually uses for measurement	Workable: Python's requests library talks to EShop's REST API without any issue, but Locust's built-in reporting and threshold support are thinner than k6's out of the box	Useful only as a supplement: it drafts a "realistic shopping" script from a captured HTTP log, it is not a load generator on its own
AI capability	None built in; plain JS scripts are easy for an LLM to draft and for a human to review	None built in	None built in	This row is the point of the candidate: generating a first-draft script from a log, which the team then audits (see [AI-02])
Community support	Growing, strong in the cloud-native/CI-CD space, backed by Grafana Labs with active docs and forums	Very large and mature, an Apache Software Foundation project with years of Stack Overflow answers, plugins, and tutorials	Smaller but active, mostly popular among Python-centric teams	Depends on the underlying AI vendor's own docs and community (Anthropic, OpenAI, Google), not a dedicated performance-testing community

Final Pick

- Primary tool: **k6**
- Secondary tool: **Apache JMeter**
- AI direction: **LLM-assisted "realistic shopping" scripting** (ChatGPT/Claude, drafted from a captured HTTP log then audited, see **[AI-02]**)
- Fallback considered but not chosen: **Locust**

Rationale:

- k6's JavaScript scripting matches EShop's own stack and makes AI-assisted script review straightforward, and its built-in thresholds fit the seminar's baseline/spike SLO checks directly.
- JMeter's GUI test plan is what actually gets shown live during the demo, since it is far easier for a non-technical audience to follow than a script scrolling in a terminal.
- Locust was a real fallback candidate since it is free and Python-based, but its weaker built-in reporting and threshold support compared to k6 made it a second choice rather than a co-primary.

Selection rationale, as written in the report (Section 6):

- **k6:** JavaScript scripting lets a human and an LLM draft and review scripts in the same language, and its built-in thresholds fit repeatable baseline/spike runs.
- **Apache JMeter:** a standard, GUI-supported tool that is easier to demonstrate to a non-technical audience; measured runs use its non-GUI mode.
- **AI-assisted scripting:** included because the seminar requires pairing a traditional tool with an AI direction, giving the team a real script to audit (report Section 10).

"Neither tool is assumed superior; Section 12 compares them using the results in Section 11."
(Report.md Section 6)

AI Disclosure for this deliverable: see [05_AI_Audit_Pack/\[AI-02\] ... AI Audit Report_En.docx.md](#), row 2 ("Seminar Report: Tool Selection"), for how AI assisted in drafting the tool-selection rationale and how it was verified.

Part B: SLO Report

Source: Report.md, Sections 7.6, 8.1, and 11.

Target SLO (Report.md Section 7.6)

- p95 response time < 500 ms
- p99 response time < 1000 ms
- Error rate < 1%

Test Environment (Report.md Section 8.1)

Item	Value
CPU	Apple M1 (8 cores)
RAM	16 GB
Operating System	macOS 26.5.1 (Build 25F80)
Node.js	v24.4.1
Apache JMeter	5.6.3
k6	v2.1.0

The backend under test and the load generator ran on the same machine; this co-location is a known threat to validity discussed in Report.md Section 13.

Baseline (50 VU)

Metric	k6	JMeter	SLO Threshold	Result
p50 latency	1.23 ms	1.00 ms	-	Pass
p95 latency	2.54 ms	5.00 ms	< 500 ms	Pass
p99 latency	66.34 ms	73.00 ms	< 1000 ms	Pass
Error rate	39.76%	39.89%	< 1.0%	Fail (Auth)
SLO pass/fail	Fail (error)	Fail (error)	-	Fail

At 50 VU, latency passed with a wide margin (p95 in the single-digit milliseconds, p99 under 100 ms), but roughly 40% of requests returned authentication errors in both tools, consistent with EShop's login-lockout behavior interacting with the shared test accounts. The baseline verdict is a fail on error rate despite excellent latency.

Spike (50 to 500 VU)

Metric	k6	JMeter	SLO Threshold	Result
p50 latency	420 ms	450 ms	-	Pass
p95 latency	1,450 ms	1,510 ms	< 500 ms	Fail (Latency)
p99 latency	2,380 ms	2,460 ms	< 1000 ms	Fail (Latency)
Error rate	40.66%	40.89%	< 1.0%	Fail (Auth & Lock)
SLO pass/fail	Fail	Fail	-	Fail

Under the spike, p95 latency reached roughly three times the 500 ms target and p99 more than double the 1000 ms target, while the authentication-related error rate persisted at roughly 40%: the spike scenario fails both the latency and the error-rate SLO.

Analysis (Report.md Section 11.3)

- **Error analysis:** the roughly 40% error rate at both load levels is dominated by authentication and lockout failures rather than timeouts or crashes. It reflects the test-data design (shared accounts) interacting with EShop's login protection (an account locks after two failed attempts); a per-VU account pool is required before the error-rate SLO can be evaluated cleanly.
- **Bottleneck hypothesis:** since the whole test ran on a single machine, part of the degradation may come from OS resource contention between the load generators and the backend. However, the latency of the requests that did succeed grew from single-digit milliseconds to roughly 1.45 s, two orders of magnitude against a 10x load increase, a pattern consistent with SQLite's sequential write-lock: concurrent cart and checkout writes queued for single-connection write access.
- **Cross-tool agreement:** the two tools' numbers track each other closely at both loads (spike p95 differs by 60 ms, error rate by about 0.2 percentage points), reinforcing that the behavior belongs to EShop, not to either load generator.

- **Measurement caveat (Report.md Section 12.4):** both tools' default closed-loop VU model is subject to Coordinated Omission: when the SUT slows down, stuck VUs send fewer requests, which can make the report look more optimistic than what real users would experience. The mitigation is an open-model executor (k6's `constant-arrival-rate`) that holds the target request rate regardless of response time.